

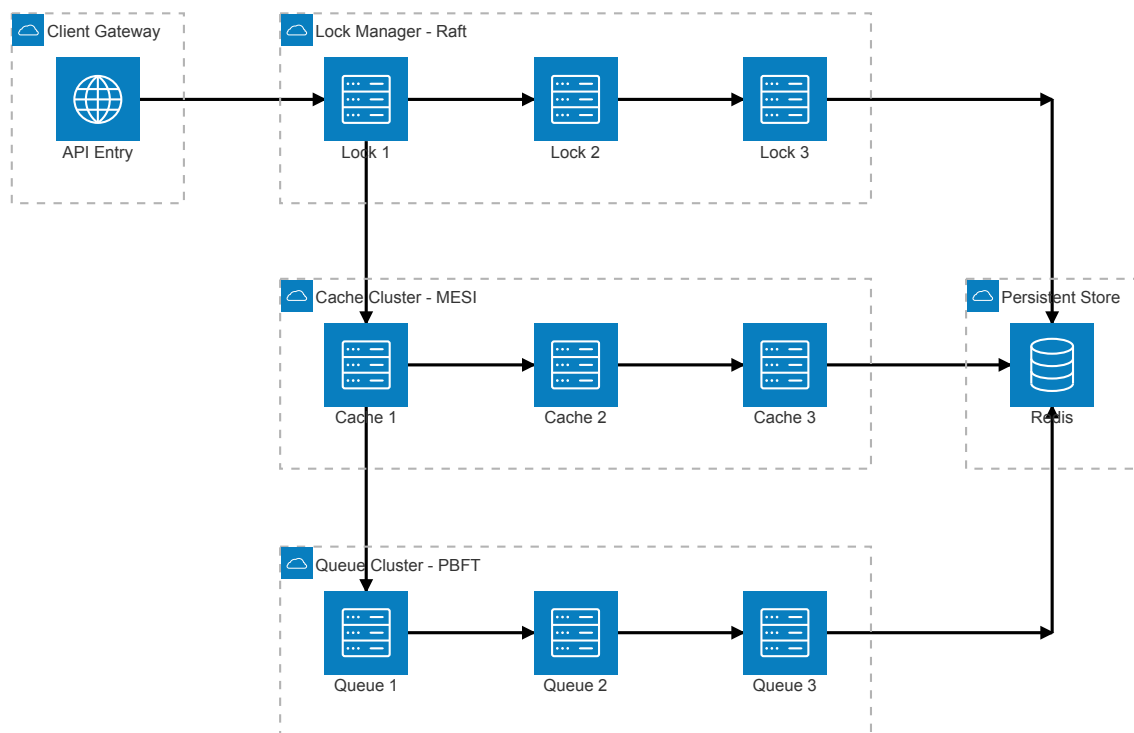
Tugas 3 - Sinkronisasi dan Distributed Systems

11231015 - Bagus Nur Andiansyah

2026-05-03

Arsitektur & Algoritma dalam Sistem

Arsitektur Sistem



Sistem yang dibangun tersusun atas beberapa cluster layanan dengan fungsi yang berbeda, yaitu *gateway*, *lock management*, *cache*, *queue*, dan *persistent storage*.

Pada lapisan paling depan, *Client Gateway* berperan sebagai titik masuk permintaan melalui layanan *API Entry*. Komponen ini menerima permintaan dari klien dan meneruskannya ke *cluster Lock Manager*. *Cluster* ini terdiri atas tiga node (**Lock 1**, **Lock 2**, dan **Lock 3**) yang menggunakan mekanisme konsensus berbasis Raft. Pola koneksi antar node yang bersifat berantai memungkinkan koordinasi replikasi log dan pemilihan pemimpin (*leader election*), sehingga konsistensi distribusi kunci dapat dijaga sebelum permintaan diproses lebih lanjut.

Setelah melewati *Lock Manager*, permintaan diteruskan ke *Cache Cluster* yang mengimplementasikan protokol MESI. Mekanisme koherensi cache berfungsi untuk menjaga konsistensi data antar replika. Alur ini memungkinkan data yang sering diakses dapat diproses dengan latensi rendah, sekaligus memastikan bahwa perubahan status cache (Modified, Exclusive, Shared, Invalid) terpropagasi secara benar sebelum interaksi dengan penyimpanan utama.

Selanjutnya, sistem meneruskan data ke *Queue Cluster* yang berbasis PBFT (Practical Byzantine Fault Tolerance). PBFT digunakan agar sistem dapat menghadapi lingkungan dengan potensi kesalahan arbitrer (*Byzantine faults*), terutama dalam pengelolaan pesan asinkron.

Pada lapisan akhir, seluruh alur berakhir pada *Persistent Store* yang memanfaatkan Redis. Terdapat dua jalur masuk menuju penyimpanan ini, yaitu melalui *junction* dari *cluster* kunci dan *cluster* antrian, serta jalur langsung dari *cluster* cache. Penyimpanan persisten berfungsi sebagai sumber kebenaran utama (*source of truth*), dengan beberapa jalur sinkronisasi untuk memastikan integritas data dari berbagai subsistem.

Algoritma

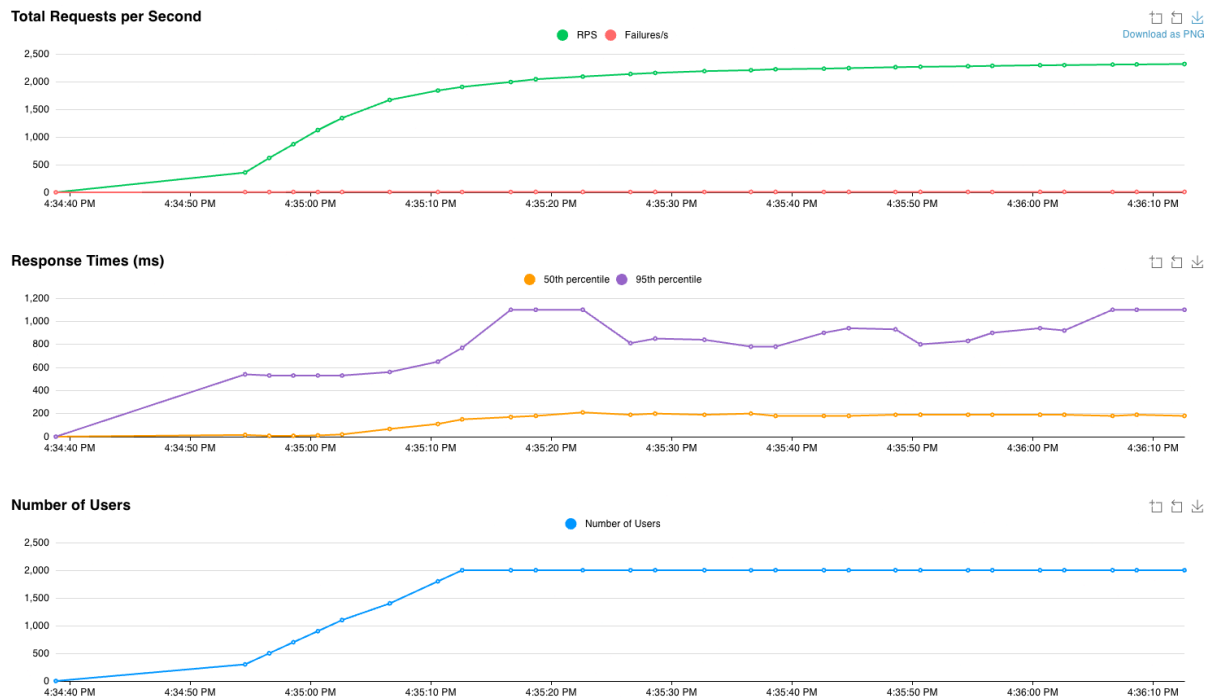
Sistem yang dibangun mengimplementasikan algoritma PBFT. *Practical Byzantine Fault Tolerance* (PBFT) sendiri merupakan algoritma konsensus yang dirancang untuk menjaga konsistensi sistem terdistribusi meskipun sebagian node mengalami kegagalan arbitrer (*Byzantine faults*), seperti memberikan informasi yang salah, tidak merespons, atau berperilaku tidak sesuai protokol. PBFT umumnya digunakan pada sistem yang membutuhkan *robustness* tanpa bergantung pada asumsi kepercayaan penuh terhadap seluruh node.

Dalam PBFT, sistem terdiri atas sejumlah replika (node) yang berperan secara kolektif untuk memproses permintaan klien. Salah satu node bertindak sebagai *primary* (pemimpin), sementara yang lain sebagai *backup*. Proses konsensus berlangsung dalam tiga tahap utama, yaitu *pre-prepare*, *prepare*, dan *commit*. Pada tahap *pre-prepare*, *primary* menerima permintaan klien dan mendistribusikannya ke seluruh *backup*. Selanjutnya, pada tahap *prepare*, setiap node memverifikasi pesan tersebut dan saling bertukar informasi untuk memastikan bahwa pesan yang diterima konsisten. Tahap terakhir, *commit*, memastikan bahwa mayoritas node telah mencapai kesepakatan sebelum eksekusi dilakukan.

Keamanan PBFT didasarkan pada prinsip kuorum. Untuk dapat mentoleransi hingga f node yang bersifat Byzantine, sistem memerlukan minimal $3f + 1$ node. Dengan demikian, konsensus dianggap tercapai jika terdapat setidaknya $2f + 1$ node yang menyetujui suatu nilai atau urutan operasi. Pendekatan ini memastikan bahwa meskipun terdapat node yang gagal atau bertindak jahat (*malicious*), hasil akhir tetap konsisten di seluruh replika yang jujur.

Salah satu keunggulan utama PBFT adalah kemampuannya mencapai *finality* secara deterministik, artinya setelah suatu keputusan diambil, tidak diperlukan konfirmasi tambahan seperti pada algoritma berbasis probabilitas. Hal ini membuat PBFT cocok untuk sistem yang membutuhkan kepastian tinggi, seperti pengelolaan antrian terdistribusi atau sistem keuangan. Namun, PBFT memiliki keterbatasan dalam hal skalabilitas, karena kompleksitas komunikasi meningkat secara kuadratik terhadap jumlah node akibat pertukaran pesan antar semua replika.

Hasil



Load testing dilaksanakan menggunakan Locust dengan total 2000 pengguna dengan *ramp-up* 100 pengguna per detik dengan tiap pengguna ditargetkan melakukan rata-rata 1 request per detik. Seiring dengan peningkatan jumlah pengguna pada fase awal, *throughput* sistem yang diukur menggunakan *requests per second* (RPS) meningkat secara signifikan dari nol hingga sekitar 2300 RPS.

Berdasarkan grafik, tingkat kegagalan (*failures per second*) relatif mendekati nol sepanjang pengujian. Hal ini menunjukkan bahwa sistem mampu mempertahankan keandalan dalam memproses permintaan, bahkan pada tingkat beban yang tinggi. Dengan demikian, dari segi *availability*, sistem mampu memberikan performa yang baik tanpa indikasi kegagalan yang berarti.

Namun demikian, grafik *response time* menggambarkan adanya degradasi kinerja dari sisi latensi. Nilai *50th percentile* meningkat dari hampir nol pada fase awal menjadi sekitar 150–200 ms setelah jumlah pengguna mencapai puncaknya. Sementara itu, *95th percentile* mengalami kenaikan yang lebih tajam, dari sekitar 500 ms hingga mendekati atau melebihi 1000 ms. Fluktuasi pada persentil ke-95 juga terlihat cukup signifikan, yang mengindikasikan adanya variasi waktu respons yang tinggi untuk sebagian permintaan. Tingginya latensi ini dapat terjadi akibat hardware yang kurang mumpuni dalam menghadapi tingginya beban request tiap detiknya.

Secara keseluruhan, hasil ini menunjukkan bahwa sistem memiliki kemampuan *scaling* yang baik dalam hal *throughput* dan *availability*, tetapi dengan kompromi pada latensi ketika beroperasi pada kapasitas maksimum.